CheckIn — Backend Report

Backend and API · Student Attendance System · IDE201 Capstone · Group 2 · College of Science and Technology

Prepared by	Dechen Wangmo (Backend and API Developer)
Date	12 October 2026
Covers	The `backend` folder (`src`, `prisma`, `scripts`) as it stands in the current branch
Stack	Node.js, Express, Prisma, PostgreSQL (Supabase), JWT, Zod

1. Summary

The CheckIn backend is a REST API. It is the only part of the system that talks to the database, and it is where every rule of the system is enforced. The frontend only shows data and collects input. It never decides who is present, who may see a record, or whether a code is still valid. The backend decides all of that.

A teacher starts a class and the backend creates a short-lived 6-character code and remembers the teacher's GPS position as the classroom centre. A student enters the code on their own device. The backend checks the code, the session, the enrolment, and the student's distance from the classroom, then records the student as Present or Flagged. Everything else (dashboards, reports, CSV and PDF exports, evidence approvals, alerts, admin tools) is built on top of those attendance records.

Item	Count
Source files in `src/`	71
Database models (tables)	11
Route handlers (including alias paths)	83
User roles	4 (student, teacher, admin, management)
Service files (business logic)	13
Validator files (input rules)	12

This report explains how the backend is built, what every folder and file is for, why it exists, and how the pieces connect. Sections 3 and 12 are the best place to start if you want the big picture.

2. Technology used

Package	What it is used for
Node.js and Express 4	The web server and the routing system
Prisma 5 (`@prisma/client`)	Reads and writes the PostgreSQL database without hand-written SQL
PostgreSQL on Supabase	The database that stores all data
`jsonwebtoken`	Login tokens (JWT) that prove who the user is
`bcryptjs`	One-way password hashing (10 rounds)
`zod`	Checks that every request body, query and URL parameter has the right shape
`helmet`	Adds standard security headers to every response
`cors`	Decides which websites may call the API
`cookie-parser`	Reads the login cookie
`express-rate-limit`	Stops brute-force and spam by limiting requests per minute
`morgan`	Prints a log line for each request
`multer`	Receives uploaded files (evidence documents and profile pictures)
`@supabase/supabase-js`	Stores uploaded files in a private Supabase Storage bucket
`pdfkit`	Builds PDF reports
`nodemailer`	Sends optional emails (only when SMTP is configured)
`dotenv`	Loads the `.env` settings file

npm scripts: `start` (run the server), `dev` (run with auto-restart), `generate` (generate the Prisma client), `seed` (load seed data), `create-admin` and `import-roster` (account tools).

3. How the backend is built

3.1 The layered idea

The backend is split into layers. Each layer has one job, and a request passes through them in the same order every time. This keeps files small and makes bugs easy to find: if a rule is wrong, it is in a service; if a URL is wrong, it is in a route.

Layer	Folder	One-line job
Entry	`server.js`, `app.js`	Start the server and assemble everything
Routes	`routes/`	Match a URL and method, and list the guards that protect it
Middleware	`middleware/`	Check the request before any logic runs (login, role, input, upload, rate limit)
Validators	`validators/`	The Zod rules that `validate` middleware applies
Controllers	`controllers/`	Take the request, call one service function, send the response
Services	`services/`	All the real rules and database work
Support	`lib/`, `utils/`, `config/`	Shared connections, small helpers, settings and constants
Data	`prisma/`	Database description and seed data
Tools	`scripts/`	One-off command-line jobs

3.2 The journey of one request

Every API call follows these steps. Taking `POST /api/attendance/checkin` as the example:

1. **`server.js`** is already running and listening on the configured port.
2. **`app.js`** receives the request and passes it through security headers, CORS, JSON parsing, cookie reading, logging and the general rate limit.
3. **`routes/attendance.routes.js`** matches the path. It lists the guards for this route: `requireAuth`, `requireRole('student')`, the check-in rate limiter and `validateBody(checkInSchema)`.
4. **`middleware/requireAuth.js`** reads the token, loads the user and sets `req.user`.
5. **`middleware/requireRole.js`** confirms the user is a student.
6. **`middleware/validate.js`** runs the Zod schema from `validators/checkin.validators.js` and replaces the body with the cleaned values.
7. **`controllers/checkin.controller.js`** calls `checkIn(req.user, req.body)`.
8. **`services/checkin.service.js`** applies the rules, using `utils/haversine.js` and `utils/code.js`, reads and writes the database through `lib/prisma.js`, and creates a notification through `services/notifications.service.js` if needed.

9. The controller sends the result as JSON. If anything threw an error along the way, **`middleware/errorHandler.js`** turns it into `{ error, code }` with the right status number.

3.3 Folder map

Folder	Files	What it holds	Why it exists
`src/config`	2	`env.js` (settings), `constants.js` (fixed values)	One place to change numbers like the geofence radius, so nothing is hard-coded in the logic
`src/lib`	3	Prisma client, mailer, file storage	The three connections to outside systems, each created once and reused
`src/middleware`	6	Auth, roles, validation, upload, rate limits, error handler	Guards that run before or after the logic
`src/utils`	11	Code generator, Haversine, JWT, passwords, CSV, PDF, errors and more	Small helpers with no database access, shared by many services
`src/validators`	12	Zod schemas, one file per feature area	Bad input is rejected before it reaches the logic
`src/routes`	11	URL tables, one per feature area	Keeps the URL list separate from the logic
`src/controllers`	11	Thin request handlers	Translate between HTTP and service functions
`src/services`	13	Business rules and database queries	The brain of the system
`prisma/`	schema, seed	Database model definition and seed loader	Lets the code use tables as JavaScript objects
`scripts/`	3	Admin account, roster import, test passwords	Jobs run from the command line, not

Folder	Files	What it holds	Why it exists
			from the API

3.4 Conventions used everywhere

- **One feature, one file in each layer.** For example, evidence documents use `relief.routes.js`, `relief.validators.js`, `relief.controller.js` and `relief.service.js`.
- **Controllers contain no rules.** Each controller line is `asyncHandler(async (req, res) => res.json(await service.something(...)))`.
- **Errors are thrown, not returned.** A service throws `NotFound(...)`, `Forbidden(...)` and so on. The error handler converts them.
- **API names differ from database names in two places.** The database stores the role `tutor`, but the API says `teacher`. The evidence feature is exposed as `/api/evidence` and `/api/relief` (both work), and is stored in the `relief_requests` table.
- **Some routes have two paths** (for example `/api/me` and `/api/profile`, or `/api/teacher` and `/api/tutor`) so the existing frontend and the design document both work.

4. Startup and configuration

4.1 `server.js`: the entry point

It builds the app with `createApp()`, starts listening on the configured port, and prints a ready message. It also listens for the stop signals `SIGINT` and `SIGTERM`. When one arrives, it stops accepting new requests, disconnects from the database, and exits cleanly. This avoids half-finished requests and leaked database connections.

4.2 `app.js`: assembling the app

`createApp()` adds the following in a fixed order. The order matters, because each step runs before the next.

1. **`trust proxy`**: the app runs behind a proxy on Render, and this is needed for correct rate limiting and secure cookies.
2. **`helmet`**: security headers.
3. **`cors`**: allows only the frontend addresses from `FRONTEND_URL` in production (any address in development), with cookies allowed.
4. **`express.json`**: reads JSON bodies up to 1 MB.
5. **`cookieParser`**: reads the login cookie.
6. **`morgan`**: request logging.
7. **`/health`**: a public status check used by hosting and pipelines. It has no login and no rate limit.
8. **`/uploads`**: serves locally saved files, used only when Supabase Storage is not configured.
9. **`apiLimiter`** on `/api`: 300 requests per minute per IP.
10. **The feature routers** (see the list below).

11. **`notFound`**: any unknown path returns 404 with a clear message.
12. **`errorHandler`**: always last, so it catches every error.

Routers mounted:

Path	Router	Who uses it
`/api/auth`	`auth.routes`	Everyone (sign-up, login)
`/api/me`, `/api/profile`	`profile.routes`	Any logged-in user
`/api/notifications`	`notifications.routes`	Any logged-in user
`/api/student`	`student.routes`	Students
`/api/modules`	`modules.routes`	Students (browse, join), teachers (key, roster)
`/api/attendance`, `/api/checkin`	`attendance.routes`	Students
`/api/sessions`	`session.routes`	Teachers
`/api/teacher`, `/api/tutor`	`tutor.routes`	Teachers
`/api/evidence`, `/api/relief`	`relief.routes`	Students and staff
`/api/admin`	`admin.routes`	Admin (changes), admin and management (reports)
`/api/management`	`management.routes`	Management and admin

4.3 `config/env.js`: settings

It reads `.env` once and exposes typed settings. Every other file imports from it and never touches `process.env` directly. This means a wrong or missing value is handled in one place.

Setting	Environment variable	Default	Meaning
Port	`PORT`	5000	Where the API listens
Allowed websites	`FRONTEND_URL`	localhost:3000 and 5173	Comma-separated list for CORS
Login token life	`JWT_EXPIRES_IN`	8h	How long a login lasts
Geofence radius	`GEOFENCE_METRES`	10	Default distance allowed from the classroom
GPS accuracy limit	`ACCURACY_FLAG_MET`	50	A worse accuracy is

Setting	Environment variable	Default	Meaning
	`RES`		flagged
Code lifetime	`CODE_VALIDITY_MINUTES`	10	How long a check-in code works
Required attendance	`REQUIRED_ATTENDANCE_PERCENT`	90	Below this a student gets a low-attendance alert
At-risk limit	`AT_RISK_ATTENDANCE_PERCENT`	80	Below this a student is shown as at risk
Re-alert gap	`LOW_ATTENDANCE_REALERT_DAYS`	7	Minimum days between repeat alerts
Storage	`SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_BUCKET`	bucket `checkin-files`	Private file storage
Email	`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `MAIL_FROM`	port 587	Optional email sending
Secret	`JWT_SECRET`	none	Signs login tokens

The JWT secret has a safety rule. If it is missing in production, the app refuses to start. In development it prints a warning and uses an insecure placeholder so the app can still run.

4.4 `config/constants.js`: fixed values

Constant	Value	Why it exists
`ATTENDED`	Present, Late	The only statuses that count towards attendance
`FLAGGED`	Flagged, Pending review	Statuses that wait for the teacher's decision
`SESSION_TYPES`	theory, practical	Allowed class types
`SESSION_STATUS`	scheduled, ongoing, ended	The life of a class session
`FLAG_REASONS`	permission_denied, outside_radius, low_accuracy	The three reasons a check-in is flagged
`FILES`	Evidence: 10 MB,	Upload limits

Constant	Value	Why it exists
	PDF/PNG/JPG/WebP. Avatar: 2 MB, PNG/JPG/WebP	
`WEEK_MS`, `DAY_MS`	Milliseconds	Used for weekly trends and date ranges

5. Database model

The file `prisma/schema.prisma` describes the 11 tables to Prisma. Each model maps to a PostgreSQL table, and column names are mapped from camelCase in code to snake_case in the database.

Model	Table	Purpose	Key points
User	`users`	Students, teachers, admin and management	Role, active flag, status, programme, department, year, avatar, notification settings; password hash may be empty for roster accounts
Program	`programs`	Programmes (also used as the department list)	Name only; has many modules
Module	`modules`	A course	Unique code, year, semester, enrolment key; belongs to a programme
Section	`sections`	Which teacher teaches a module	Links module and teacher, with section name, room and schedule
ClassSession	`class_sessions`	One class that is held	Date, start time, duration in minutes, type, status
AttendanceCode	`attendance_codes`	The check-in code for a session	Expiry time, geofence radius and the classroom latitude and longitude
Attendance	`attendance`	One row per student per session	Status, submitted position, accuracy, distance, flag reason

Model	Table	Purpose	Key points
Enrollment	`enrollments`	Which student joined which module	Composite key of student and module
EvidenceDocument	`relief_requests`	Absence evidence requests	Date, leave type, document path, status, reviewer
Notification	`notifications`	In-app messages	Message, category, read flag
AuditLog	`audit_logs`	Record of admin actions	Admin, action, details, time

5.1 How the tables relate

- A **Program** has many **Modules**.
- A **Module** has one or more **Sections**, and each Section is taught by one **User** (a teacher).
- A **Section** has many **ClassSessions**, and each session has one or more **AttendanceCodes** (a new one each time the code is regenerated).
- A **ClassSession** has many **Attendance** rows, one per student.
- A **User** (student) joins Modules through **Enrollment**.
- A student submits **EvidenceDocument** rows for a Module, and a staff **User** reviews them.
- Every **Notification** belongs to one User. Every **AuditLog** belongs to one admin User.

5.2 Rules the database itself enforces

- **One attendance row per student per session** (`@@unique([studentId, sessionId])`). This is what stops a student from checking in twice, even if they tap twice at the same instant.
- **One enrolment per student per module** (composite primary key).
- **A code belongs to one session** (composite key of session and code).
- **Unique email** for users and **unique module code**.
- **Delete rules**: deleting a user or module that still has attendance, sessions or evidence is blocked (`Restrict`). Enrolments and notifications are deleted with their parent (`Cascade`). A reviewer who is deleted leaves the evidence record with an empty reviewer (`SetNull`).
- **Readable IDs**: most tables generate IDs like `P001`, `M001`, `S001`, `A001` through a database function `gen_id()` and sequences. User IDs are the college ID typed by the person (for example `02250368` or `STF-001`).

5.3 Time values

A session stores its date and its start time in two separate columns. The helper `sessionStart()` in `utils/format.js` joins them into one UTC moment. All time comparisons (is this session already held?) use that joined value.

6. Shared building blocks

6.1 `lib/`: connections to outside systems

File	What it does	How it is built	Why it exists
`prisma.js`	Creates the one Prisma client	Stored on a global so `node --watch` restarts reuse it	Opening many clients would use up database connections
`storage.js`	Saves, opens and deletes uploaded files	If Supabase settings exist, files go to a private bucket and the database keeps `storage:<path>`. Otherwise files go to `backend/uploads/`. `fileUrl()` returns a one-hour signed link for private files	Evidence documents are private, so they must not have a public address
`mailer.js`	Sends optional emails	Builds the transport only when first needed. If `SMTP_HOST` is empty, it silently skips. `sendMailInBackground` never blocks or breaks the request	Email is a bonus. The in-app notification always works

6.2 `middleware/`: guards

File	What it does	Details
`requireAuth.js`	Proves who the user is	Reads the token from the `Authorization: Bearer` header or the `checkin_token` cookie, verifies it, then loads the user from the database. If the user is missing or deactivated, access is refused immediately, not only when the token expires. Sets `req.user` as id, role, name, email. `identify()` does the same without throwing and is used by `/auth/me`

File	What it does	Details
`requireRole.js`	Checks the role	`requireRole('teacher')` or `requireRole('admin', 'management')`. Returns 403 `ROLE_FORBIDDEN` otherwise
`validate.js`	Applies Zod rules	Three versions for body, query and params. On success the cleaned value replaces the original (numbers become numbers, text is trimmed). On failure it returns 400 `VALIDATION_ERROR` with the first problem
`upload.js`	Receives files	Multer keeps the file in memory and enforces size and type from `constants.js`. Two ready-made uploaders: evidence and avatar
`rateLimits.js`	Slows abuse	General API: 300 per minute. Login and sign-up: 15 failed tries per minute (successful logins do not count). ID lookup: 10 per minute (stops scanning student numbers). Check-in: 20 per minute (stops code guessing)
`errorHandler.js`	Final safety net	Converts `HttpError`, Prisma errors (duplicate 409, missing 404, bad reference 400), upload errors and bad JSON into `{ error, code }`. Unknown errors become 500 and show the real message only outside production

6.3 `utils/`: helpers

File	What it does
`errors.js`	The `HttpError` class, shortcuts `BadRequest`, `Unauthorized`, `Forbidden`, `NotFound`, `Conflict`, and `asyncHandler`, which sends a

File	What it does
	failed promise to the error handler
`format.js`	Converts database rows to API shapes: `roleOut`/`roleIn` (tutor and teacher), `apiStatus` (any status becomes present, flagged or absent), `sessionStart`, `round1`, `userOut` (a safe user object that never contains the password hash), and the default notification settings
`code.js`	Generates the 6-character check-in code using a secure random generator. The alphabet leaves out look-alike characters (0, O, 1, I, L) so the code is easy to read from a projector. It also cleans typed codes (trim, uppercase, remove spaces)
`haversine.js`	Distance in metres between two GPS points on the Earth, using the Haversine formula with an Earth radius of 6,371,000 m
`jwt.js`	Signs and verifies login tokens. The payload is the user id and role
`password.js`	Hashes with bcrypt (10 rounds), verifies, and defines the password rule: at least 8 characters, a number and a symbol. Also holds a dummy hash so a login for a missing user takes as long as a wrong password
`cookies.js`	Sets and clears the httpOnly login cookie (8 hours, secure and cross-site in production)
`csv.js`	Builds CSV text with correct quoting and a byte-order mark so Excel reads names correctly
`pdf.js`	PDF building blocks on pdfkit: header, section titles, tables that break across pages, and a footer with page numbers
`enrolment-key.js`	Suggests a key like `IDE201-3-T1-K7Q`, validates a key (3 to 30 characters: letters, digits, dash, underscore) and normalises it to uppercase
`id-format.js`	Trims and uppercases IDs so lookups are forgiving

6.4 `validators/`: input rules

Each file defines Zod schemas for one feature. They are small on purpose and are applied by `validate.js`. `common.js` holds the reusable pieces: a required text ID, optional text, optional numbers (empty form fields count as missing), latitude and longitude ranges, a date as `YYYY-MM-DD`, and a month as `YYYY-MM`.

File	What it checks
`auth.validators.js`	Register form, login, ID lookup, roster sign-up, change password (accepts both `current/next` and `currentPassword/newPassword`). Passwords must meet the strong-password rule
`checkin.validators.js`	Code of 4 to 12 characters, optional latitude, longitude and accuracy, optional `permissionDenied`
`session.validators.js`	Start a session: a module or an existing session, type, hours (0.5 to 6), required latitude and longitude, optional code minutes (1 to 120) and radius (5 to 500 m). Also flagged decision parameters and session list filters
`module.validators.js`	Module id, enrolment key rules, add-student body
`relief.validators.js`	Evidence submission (module, date, type medical or official, reason up to 500 characters), review decision, list filter
`admin.validators.js`	Programme names, user filters and patch, module form, at-risk filters, audit-log filters
`management.validators.js`	Department name, ids, history filters, calendar parameters
`profile.validators.js`	Name, email, phone, notification settings
`notification.validators.js`	Notification id and list filters
`attendance.validators.js`	Month for the student calendar (defaults to the current month)
`tutor.validators.js`	Module id and student id parameters for teacher records

7. Features and logic

This section explains each feature in the order a real class day happens. For every feature it names the service file that holds the rules.

7.1 Accounts and login (`auth.service.js`)

There are two ways to get an account.

A. Roster sign-up. An admin pre-loads the college roster with `scripts/import-roster.js`. These people exist in the database with no password.

1. The person types their ID. `lookupId` trims and uppercases it, then checks the roster. Unknown ID returns 404 `ID_NOT_FOUND`. An ID that already has a password returns 409 `ALREADY_SIGNED_UP`. A deactivated one returns 403 `ACCOUNT_INACTIVE`. Otherwise the name, email, role, programme and department are shown back to them.
2. They choose a password. `claimRosterAccount` checks that the ID belongs to the right role, hashes the password and saves it. A student becomes active at once and is logged in. A teacher stays inactive and waits for approval, and every active admin gets a notification.

B. Open register form. The person fills in everything. If their ID is on the roster without a password, the system switches to the roster flow instead of failing. If the ID already has an account, it returns 409. Otherwise a new user is created. A student is active straight away and starts in year 1. A teacher is created inactive (pending). If no department is given, it is worked out from the programme name by removing the leading "BE" or "B.E.".

Login. The service always runs the password check, even when the ID does not exist (it compares against a dummy hash). That way, a missing user and a wrong password take the same time and give the same message (401 `INVALID_CREDENTIALS`), so nobody can discover which IDs exist. An inactive account returns 403: `ACCOUNT_PENDING` for a teacher waiting for approval, or `ACCOUNT_NOT_ACTIVE` otherwise. On success the token is returned and the controller also sets the httpOnly cookie.

Account state	How the database shows it	Can log in?
Student, active	`isActive = true`, `status = active`	Yes
Teacher waiting for approval	`isActive = false`, `status = active`, has a password	No (`ACCOUNT_PENDING`)
Teacher approved	`isActive = true`	Yes
Teacher rejected or user deactivated	`status = inactive`	No
Roster account, not yet signed up	No password hash	No, must sign up first

Other account functions. `getMe` returns the safe user object (and the route `/api/auth/me` answers with `null`, not an error, when logged out, which is handy on page load). `changePassword` requires the current password to be correct and the new one to be different and strong.

7.2 Student check-in (`checkin.service.js`)

This is the core feature. The function `checkIn(student, { code, lat, lng, accuracy, permissionDenied })` works through these steps and stops at the first failure.

1. **Clean and format-check the code.** It is trimmed, uppercased and must be 6 letters or digits. Otherwise 400 `CODE_INVALID`.
2. **Find the code.** The most recently expiring matching code is loaded together with its session, section and module. Not found returns 400 `CODE_INVALID`.
3. **Session not ended.** An ended session returns 400 `SESSION_ENDED`.
4. **Code not expired.** A code past its expiry time returns 400 `CODE_EXPIRED`.
5. **Student is enrolled** in the module. Otherwise 403 `NOT_ENROLLED`.
6. **Location decision** (below).
7. **Save the attendance row.** The database rule that allows one row per student per session is the check for duplicates. A second attempt returns 409 `ALREADY_CHECKED_IN`, even when two taps arrive at the same moment.
8. **Notify the teacher** if the check-in was flagged.

Situation	Result	Flag reason saved
No location sent, or the student denied permission	Flagged	`permission_denied`
Distance to the classroom is larger than the code's geofence radius	Flagged	`outside_radius`
Inside the radius, but GPS accuracy is worse than 50 m	Flagged	`low_accuracy`
Inside the radius with good accuracy	Present	none

The distance comes from `haversineMeters()` between the student's position and the classroom position that was stored on the code when the teacher started the session. It is rounded to one decimal and saved on the attendance row, together with the submitted position and the accuracy, so the teacher can review it later.

A flagged check-in does not count towards attendance. The teacher is told with a message such as "Name checked in 45 m from the classroom (CODE). Please verify.", and the student sees: "Checked in, but your location could not be confirmed. Your tutor will review it."

The route also has its own rate limiter (20 attempts per minute) so a code cannot be guessed by brute force.

7.3 Teacher sessions (`session.service.js`)

All routes here are teacher-only.

Ownership guard. `ownedSession(id, tutorId)` loads a session and confirms this teacher teaches it. If not, it answers "not found" (404) rather than "forbidden", so a teacher cannot even learn that another teacher's session exists.

Unique code. `uniqueCode()` generates a code and checks that no other unexpired code is identical. It tries up to 10 times.

Start attendance (`startSession`). It accepts either:

- a **module** (plus type, hours, optional room), which creates a new session for now with status `ongoing` and a first code, or
- a **sessionId**, which re-opens an existing session. If it already has a live code, that same code is returned. Otherwise a fresh code is created and the session becomes `ongoing`. An ended session returns 409.

The teacher's current GPS position (`lat`, `lng`) becomes the classroom centre stored on the code. The code lifetime defaults to the `CODE_VALIDITY_MINUTES` setting (10) and the radius to `GEOFENCE_METRES` (10). Both can be overridden per request (1 to 120 minutes, 5 to 500 m). If a room is given and differs from the section's room, the section's room is updated. The reply contains the session id, code, expiry time, radius and minutes.

Live counter (`live`). Polled by the Start Attendance page. It returns the code, expiry, session status, enrolled total, how many have checked in (Present or Late), how many are absent so far (enrolled minus the number of rows), and the list of flagged students with their flag reason.

Roster (`roster`). Starts from the enrolment list, so students who never checked in still appear as absent. Each row has status, time, flag reason, distance and accuracy. The reply also includes counts for enrolled, present, flagged and absent.

End session (`endSession`). In one transaction the session is marked `ended` and every still-valid code is expired, so the code stops working immediately.

Decide a flagged check-in (`decideFlagged`). The teacher accepts (becomes Present) or rejects (becomes Absent). The row must belong to this session and must currently be flagged. An optional note is appended to the flag reason (limited to 100 characters). The student is notified.

Mark manually (`markManually`). For cases such as a dead phone. The student must be enrolled. A row is created or overwritten as Present or Absent with the reason `marked_by_tutor`.

List my sessions (`listMine`). The teacher's sessions, newest first, with module, room, date, hours, type, status and number of check-ins.

Session status	Meaning
`scheduled`	Exists but attendance has not been opened
`ongoing`	A code has been generated

Session status	Meaning
`ended`	Closed by the teacher, codes expired

7.4 Attendance maths (`stats.service.js`)

This file is the single place where percentages are calculated. Every dashboard, report, export and alert calls it, so the numbers always agree with each other.

Formula (hour-weighted):

> *attendance % = hours attended (Present or Late) ÷ hours of sessions already held × 100*

- A session counts as **held** once its start time has passed.
- A session's **hours** are its duration in minutes divided by 60 (a missing duration counts as 1 hour). A 2-hour practical therefore counts twice as much as a 1-hour lecture.
- A student with **no record** for a held session is **absent**. Absence is worked out when the numbers are read, so no job is needed to create absent rows.
- **Flagged check-ins do not count** until the teacher accepts them.
- The **overall** figure across several modules is hour-weighted, not an average of averages.
- When no session has been held, the percentage is empty (`null`), which the screens show as "no sessions" or 0.

Example: a module has one 1-hour theory class and one 2-hour practical, and the student attended only the theory class. Hours attended 1, hours held 3, so 33.3%.

Main functions:

Function	What it returns
`loadMatrix(moduleIds, studentIds)`	Loads sessions and attended records in just two queries, then answers questions from memory: `pct`, `tally`, `overall`, `sessionsOf`, `attendedSession`. Used by every list and report so a page does not run hundreds of queries
`modulePct`, `overallPct`	One student's percentage in one module, or overall
`overallFor(studentIds)`	A map of overall percentage for many students at once
`monthDays(studentId, moduleIds, month)`	The calendar: for each day, present, flagged or absent. When a day has several sessions, the best status wins
`classRate(sessions, classSize)`	Class-level rate: attended records ÷ (sessions ×

Function	What it returns
	class size)
`heldSessions`, `isHeld`, `hoursOf`, `enrolledModuleIds`	Small helpers

7.5 Student screens (`student.service.js`)

Function	Route	What it returns
`dashboard`	`GET /api/student/dashboard`	One card per enrolled module: percentage, `atRisk` (below 80) and `belowRequired` (below 90)
`summary`	`GET /api/student/summary`	Overall rate, number of sessions missed, hours missed, and the required percentage
`moduleRecords`	`GET /api/student/modules/:id/records`	Session-by-session list, newest first, split into theory and practical with a percentage for each. Returns 403 if not enrolled
`browseModules`	`GET /api/modules/browse`	Every module with department, year, semester, number of students and whether the student has joined. **The enrolment key is never sent**
`enrol`	`POST /api/modules/enrol`	Joins a module using the key. The match ignores upper and lower case. Unknown key returns 404 `KEY_NOT_FOUND`. Already joined returns 409 `ALREADY_ENROLLED`
calendar	`GET /api/attendance/me?month=2026-10`	The monthly calendar from `monthDays`

7.6 Teacher screens (`tutor.service.js`)

Function	What it does
`myModules`	The teacher's modules (one entry per module even if they teach two sections), with student

Function	What it does
	count, room, schedule and the enrolment key
`dashboard`	Totals (modules, distinct students, flagged check-ins waiting, pending evidence requests), today's sessions with check-in counts, and the module list
`moduleRecords`	One row per student with theory %, practical % and total %. Checks the teacher owns the module first
`studentRecord`	One student in one module, one row per day with theory and practical status
`reports`	Last 6 weeks of class attendance for each module, plus an overall figure, for the weekly chart
`exportRows`, `csvRows`, `CSV_HEADER`	Prepare the export table. Status is "At risk" below 80, "OK" otherwise, or "No sessions"
`writeRecordsPdf`	Streams one module's report as a PDF
`writeModulesPdf`	Streams one PDF covering all of the teacher's modules, with the average per module

CSV downloads use `utils/csv.js` (with the Excel-friendly byte-order mark). PDFs use `utils/pdf.js`. In `tutor.routes.js` the export routes are placed before `/records/:moduleId/:studentId` on purpose, so the words "csv" and "pdf" are never mistaken for a student ID.

7.7 Modules and enrolment (`module.service.js`)

The teacher shares an **enrolment key**, and students join with it (similar to Moodle).

Function	What it does
`assertTutorOwnsModule`	The guard used by every teacher route. Returns the teacher's section for the module or a 403 `NOT_YOUR_MODULE`
`suggestKey`	Proposes a key such as `IDE201-3-T1-K7Q`
`setEnrolmentKey`	Saves a key after the format rule passes. Keys are stored in uppercase and must be unique across all modules, ignoring case, otherwise `KEY_TAKEN`
`listStudents`	Students in a module. Teachers see only their own modules. Admin and management can see any

Function	What it does
`addStudent`	The teacher adds a student by ID. If already enrolled, nothing changes
`removeStudent`	The teacher removes a student

7.8 Evidence documents (`relief.service.js`)

A student who missed a class uploads a medical or official leave document. A teacher of that module, an admin, or management can approve it.

Submit. The student must be enrolled in the module. The date cannot be in the future. A request for the same module and date that is already Pending or Approved is refused (409 `DUPLICATE_REQUEST`). The file (PDF, PNG, JPG or WebP, up to 10 MB) is saved under `relief/<studentId>/` through `lib/storage.js`. Every teacher of the module is notified.

Who can see what (`canSee`): a student sees only their own requests; a teacher sees requests for modules they teach; admin and management see all.

Review. The request must be Pending (otherwise 409 `ALREADY_REVIEWED`). When it is **approved**, every session of that module on that date is set to Present for that student, with the flag reason `relief_approved`. The status change and all attendance updates run in one transaction, so they succeed or fail together. The student is notified (and emailed if email is set up). Reviews by admin or management are also written to the audit log.

Viewing the document. The API returns a one-hour signed link, never the permanent storage path.

Status	Meaning
Pending	Waiting for a decision
Approved	Sessions that day were set to Present
Rejected	No change to attendance

7.9 Notifications (`notifications.service.js`)

Notifications are saved in the database and shown in the bell on the screens. They are created by other services whenever something needs a person's attention.

Function	What it does
`notify(userId, message, category, { email })`	Saves a notification. If an email subject is given, the user has an email address and has not turned email off in their settings, an email is also sent in the background. **It never throws**, so a failed notification can never break the main action
`notifyAdmins(message)`	Sends the same message to every active admin

Function	What it does
`list`	A user's notifications, newest first (default 50), optionally unread only
`unreadCount`	The number for the bell badge
`markRead`	Marks one notification read. The query includes the user's own id, so nobody can mark someone else's
`markAllRead`	Marks all of a user's unread notifications read

Where notifications come from: a flagged check-in (to the teacher), a new evidence request (to the module's teachers), an evidence decision (to the student), a flagged check-in decision (to the student), a teacher account request (to admins), a tutor approval (to the tutor), a module assignment (to the tutor), and the low-attendance alert (to the student).

7.10 Profile (`profile.service.js`)

Function	What it does
`getProfile`	The logged-in user's details, with a usable picture link
`updateProfile`	Changes name, email, phone and notification switches. Only the fields that were sent are changed, so sending a phone number alone does not erase the name. The notification switches are merged with the existing ones
`setAvatar`	Saves a new profile picture (PNG, JPG or WebP, up to 2 MB) under `avatars/<userId>`, updates the user, then deletes the old picture
`removeAvatar`	Clears the picture and deletes the file

The default notification settings are low-attendance alerts on, evidence updates on, reminders off, email on, SMS off, push on.

7.11 Admin (`admin.service.js`)

Changes are admin-only. Reports are shared with management. Every change is written to the audit log.

Function	What it does
`overview`	Counts: students, students who joined this month, active teachers, teachers waiting for approval, programmes, modules

Function	What it does
`listPrograms`, `createProgram`, `renameProgram`	Programme list. A new name must not already exist (ignoring case). Renaming also updates the department text of every user that used the old name, in one transaction
`listUsers`	Users filtered by role, department and year, with each student's overall percentage. Older accounts with no year are treated as year 1
`patchUser`	Activate, deactivate or rename a user. An admin cannot deactivate their own account
`listModules`, `saveModule`	List modules, or create or update one. If the programme name is new, it is created. If a teacher ID is given, the module's first section is assigned to that teacher (or created as section A) inside one transaction, and the teacher is notified. This assignment is what makes the module appear on the teacher's screens
`pendingTutors`, `approveTutor`, `rejectTutor`	The approval queue (a teacher who signed up, has a password and is not yet active). Approve activates the account, notifies the person and emails them. Reject marks the account inactive
`atRisk`	Every student and module pair below the limit (default 80), worst first. It works per module so that a weak module is not hidden by a good average
`programmeReport`	Every student in a programme with their overall percentage and an At risk or OK label, plus the average
`runLowAttendanceJob`	The alert job (below)

The low-attendance alert job. Triggered by the admin button (`POST /api/admin/jobs/low-attendance`). For every enrolled, active student, if the percentage in a module is below the required level (default 90), the student gets a notification and an email. A student who was already alerted about the same module within the last 7 days is skipped, so the job is safe to run as often as wanted. It returns the number scanned, alerted and skipped. There is no built-in scheduler, so the job runs only when someone triggers it (or when a timer or cron is added).

7.12 Management dashboards (`management.service.js`)

Read-only views for the Dean of Academic Affairs. Admin can also see them. They drill down from college to department to module to student. In this system a department is a programme name.

Function	What it returns
`overview`	College totals and one row per department: students, modules and attendance rate. "Below 80" counts the departments under the at-risk limit
`department`	Student count, average, number at risk, a 6-week trend, and every module with its rate
`moduleView`	Students with their absences, hours missed and percentage, plus total absences and approved evidence requests
`studentView`	One student's history in one module: present, absent, total, percentage
`teachers`	Teachers, their modules, class names and total hours
`history`	A searchable list of student and module pairs with the percentage and teacher
`calendar`	Calendar days for one student in one module

The department and module rates use a class-level method: attended records divided by (sessions held × class size). The 6-week trend repeats the same calculation for each week.

7.13 Audit log (`audit.service.js`)

`writeAuditLog(adminId, action, details)` records who did what. It never throws (a failed log never blocks the action) and keeps at most 500 characters of detail. Actions recorded include: created or renamed a programme, edited, activated or deactivated a user, created or updated a module, approved or rejected a tutor, ran the low-attendance job, and relief approved or rejected by admin or management. `listAuditLogs` filters by action, date range (the whole end day is included), text, and limit (up to 500). `listAuditActions` lists the distinct action names for the filter box.

8. API overview

The full route list with request and response notes is in `backend/API.md`. This table shows how each URL area maps to the files that serve it.

Base path	Who may call	Route file	Controller	Service
`/api/auth`	Public (some	`auth.routes`	`auth.controll`	`auth.service`

Base path	Who may call	Route file	Controller	Service
	routes need login)		er	
/api/me, /api/profile	Any logged-in user	profile.routes	profile.controller	profile.service
/api/notifications	Any logged-in user	notifications.routes	notifications.controller	notifications.service
/api/student	Student	student.routes	student.controller	student.service, stats.service
/api/modules	Student (browse, enrol), teacher (key, roster), admin and management (view students)	modules.routes	module.controller, student.controller	module.service, student.service
/api/attendance, /api/checkin	Student	attendance.routes	checkin.controller, student.controller	checkin.service, stats.service
/api/sessions	Teacher	session.routes	session.controller	session.service
/api/teacher, /api/tutor	Teacher	tutor.routes	tutor.controller	tutor.service
/api/evidence, /api/relief	Student (submit, own list), teacher, admin and management (review)	relief.routes	relief.controller	relief.service
/api/admin	Admin (changes), admin and management (reports)	admin.routes	admin.controller	admin.service, audit.service
/api/management	Management and admin	management.routes	management.controller	management.service

How a route file is built. Each one creates an Express `Router`, applies the guards that cover the whole file with `router.use(requireAuth, requireRole(...))`, then lists routes in the form `router.get(path, extra guards, validators, controller)`. Where one path could

swallow another, the fixed path is placed first (for example `/unread-count` before `/:id` in notifications, `/mine` before `/:id` in evidence).

How a controller is built. It is one line per route: wrap in `asyncHandler`, call the service with `req.user`, `req.params`, `req.query` or `req.body`, and return JSON. Controllers for downloads (`tutor.controller`, `admin.controller`) call `sendCsv` or stream a PDF instead.

9. Security

Protection	How it works	Where
Passwords	Hashed with bcrypt (10 rounds). The hash is never returned by the API	`utils/password.js`, `utils/format.js`
Strong passwords	At least 8 characters, a number and a symbol	`auth.validators.js`
Login token	JWT with user id and role, valid 8 hours	`utils/jwt.js`
Cookie	httpOnly, so page scripts cannot read it. Secure and cross-site in production	`utils/cookies.js`
Instant deactivation	The user is reloaded on every request, so a deactivated account loses access at once	`requireAuth.js`
Role checks	Every route declares which roles may use it	`requireRole.js`, route files
Ownership checks	A teacher can reach only their own modules and sessions. Other teachers' sessions answer "not found"	`module.service.js`, `session.service.js`
Student privacy	A student can see only their own records and requests. The enrolment key is never sent to students	`student.service.js`, `relief.service.js`
Login does not leak accounts	Same time and message for unknown ID and wrong password	`auth.service.js`
Rate limits	300 per minute overall, 15 failed logins, 10 ID lookups, 20	`rateLimits.js`

Protection	How it works	Where
	check-ins per minute	
Input validation	Every body, query and parameter is checked by Zod before use	`validators/`, `validate.js`
Upload limits	Type and size are checked. Files are kept in memory, then stored privately	`upload.js`, `storage.js`
Private files	Evidence documents are in a private bucket and opened with one-hour signed links	`storage.js`
Security headers	`helmet` on every response	`app.js`
CORS	Only the listed frontend addresses in production	`app.js`, `env.js`
Secrets	Read from `.env`, which is not committed. The app refuses to start in production without a JWT secret	`env.js`
Error hiding	In production, unexpected errors return a generic message	`errorHandler.js`
Admin accountability	Admin actions are written to the audit log	`audit.service.js`

10. Scripts and seed data

File	How to run	What it does
`scripts/create-admin.js`	`node scripts/create-admin.js <userId> <email> "<name>" <password> [admin or management]`	Creates or resets an admin or management account. Checks the password rule, hashes it, and activates the account. This is how admin and management accounts exist, because they cannot self-register
`scripts/import-roster.js`	`node scripts/import-roster.js roster.csv`	Reads a CSV with the columns `userId,name,email,role,programme,department,year` using a built-in parser that understands quoted fields.

File	How to run	What it does
		Creates users with no password so they can sign up by ID. Existing users are left alone. `roster.example.csv` shows the format
`scripts/set-test-passwords.js`	`node scripts/set-test-passwords.js <password>`	Sets one password for every user. **It has no filter**, so it overwrites all accounts, including admin. Use only on a local test database
`prisma/seed.js`	`npm run seed`	Takes the SEED DATA part of `prisma/database.sql` and runs it in one transaction. Safe to run again, because existing rows are skipped
`prisma/database.sql`	Run in the Supabase SQL Editor	Structure changes plus seed data in one re-runnable file. It was not part of the code export used for this report, so check it against the schema before quoting its contents

11. Error handling

Every error reaches the one `errorHandler` and leaves the API in the same shape, so the frontend can always show `error` to the user:

```
{ "error": "message for the user", "code": "SOME_CODE" }
```

Status	Typical codes	When
400	`VALIDATION_ERROR`, `CODE_INVALID`, `CODE_EXPIRED`, `SESSION_ENDED`, `FUTURE_DATE`, `FILE_TYPE`, `UPLOAD_ERROR`, `BAD_JSON`	Bad input or an invalid action
401	`UNAUTHORIZED`, `INVALID_CREDENTIALS`	Not logged in, or wrong ID or password
403	`ROLE_FORBIDDEN`, `NOT_ENROLLED`,	The user is not allowed

Status	Typical codes	When
	`NOT_YOUR_MODULE`, `ACCOUNT_PENDING`, `ACCOUNT_NOT_ACTIVE`, `KEY_TAKEN`	
404	`ROUTE_NOT_FOUND`, `NOT_FOUND`, `ID_NOT_FOUND`, `KEY_NOT_FOUND`, `SESSION_NOT_FOUND`	Something does not exist
409	`ALREADY_CHECKED_IN`, `ALREADY_ENROLLED`, `ALREADY_SIGNED_UP`, `DUPLICATE_REQUEST`, `ALREADY_REVIEWED`, `DUPLICATE`	A conflict with existing data
429	`RATE_LIMITED`	Too many requests
500	`INTERNAL`	An unexpected error. The real message is hidden in production

The handler also recognises Prisma errors (`P2002` duplicate, `P2025` not found, `P2003` bad reference) and upload errors such as a file that is too large.

12. How everything connects

12.1 Feature to files

This table shows which files work together for each feature, in the order a request passes through them.

Feature	Route and guards	Validator	Controller	Service	Helpers and data
Login and sign-up	`auth.routes` with `authLimiter`	`auth.validators`	`auth.controller`	`auth.service`, `notifications.service`	`password`, `jwt`, `cookies`, `id-format`, tables `users`, `notifications`
Start attendance	`session.routes` (`requireAuth`, `requireRole`	`session.validators`	`session.controller`	`session.service`, `module.service`	`code`, tables `class_sessions`, `attendance_codes`,

Feature	Route and guards	Validator	Controller	Service	Helpers and data
	`('teacher')`)				`sections`
Student check-in	`attendance.routes` (student, `checkinLimiter`)	`checkin.validators`	`checkin.controller`	`checkin.service`, `notifications.service`	`haversine`, `code`, `constants`, tables `attendance_codes`, `enrollments`, `attendance`
Flagged review and manual marks	`session.routes`	`session.validators`	`session.controller`	`session.service`, `notifications.service`	table `attendance`
Percentages and dashboards	`student.routes`, `tutor.routes`, `management.routes`	`attendance`, `tutor`, `management` validators	matching controllers	`stats.service` with `student`, `tutor`, `management` services	`format`, tables `class_sessions`, `attendance`, `enrollments`
Enrolment	`modules.routes`	`module.validators`	`module.controller`, `student.controller`	`module.service`, `student.service`	`enrolment-key`, table `enrollments`
Evidence documents	`relief.routes` with `upload`	`relief.validators`	`relief.controller`	`relief.service`, `audit.service`, `notifications.service`	`storage`, tables `relief_requests`, `attendance`
Exports	`tutor.routes`, `admin.routes`	`tutor`, `admin` validators	`tutor.controller`, `admin.controller`	`tutor.service`, `admin.service`	`csv`, `pdf`
Admin tools	`admin.routes`	`admin.validators`	`admin.controller`	`admin.service`, `audit.service`	tables `users`, `modules`, `programs`, `audit_logs`
Alerts	`admin.route`	none	`admin.contr`	`admin.servi`	`mailer`, table

Feature	Route and guards	Validator	Controller	Service	Helpers and data
	s (POST /jobs/low-attendance)		oller	ce, stats.service, notifications.service	notifications
Profile and pictures	profile.routes with upload	profile.validators	profile.controller	profile.service	storage, table users

12.2 Which files depend on which

- **lib/prisma.js** is used by every service and by requireAuth. It is the only door to the database.
- **stats.service.js** is used by the student, teacher, admin and management services. This is why all percentages agree.
- **notifications.service.js** is used by the check-in, session, evidence, admin and auth services. It uses lib/mailer.js for optional email.
- **audit.service.js** is used by the admin and evidence services.
- **module.service.js** provides assertTutorOwnsModule, which the session and teacher services reuse. This is the single ownership rule for teachers.
- **lib/storage.js** is used by the evidence and profile services.
- **config/env.js** and **config/constants.js** are read by almost everything. Nothing else reads process.env.
- **utils/errors.js** is used everywhere, so a thrown error always becomes a clean reply.
- **utils/format.js** is used by services and middleware to convert database values to API values (for example tutor to teacher, any status to present, flagged or absent).

12.3 Worked example: one class from start to finish

1. **The teacher starts attendance.** The page sends the module, type, hours, room and the teacher's GPS position to POST /api/sessions. The route checks login, role and input. session.service.startSession confirms the teacher owns the module, generates a unique 6-character code, creates the session as ongoing, and stores the code with its expiry, radius and the teacher's position. The reply contains the code and expiry time, which the page shows with a countdown.
2. **Students check in.** Each student enters the code. checkin.service.checkIn checks the code, session, expiry and enrolment, measures the distance with the Haversine formula, and saves Present or Flagged.
3. **The teacher watches live.** The page repeatedly calls GET /api/sessions/:id/live and shows checked-in, absent and flagged counts. For the full list it calls GET /api/sessions/:id/roster.

4. **The teacher resolves flagged check-ins.** `POST /api/sessions/:id/flagged/:attendanceId/accept` or `/reject` changes the status, and the student is notified. A student who could not check in at all can be marked with `.../students/:studentId/mark`.
5. **The teacher ends the session.** `POST /api/sessions/:id/end` marks it `ended` and expires the code. Students who have no attendance row for this held session now count as absent, because `stats.service` treats a missing record as absent.
6. **Everyone sees the result.** Student dashboard, teacher records and reports, management views and exports all read the same data through `stats.service`.

12.4 Worked example: an absence with evidence

A student uploads a document through `POST /api/evidence`. Multer receives the file first (so the form fields exist), then the validator checks the fields, then `relief.service.submit` checks enrolment and duplicates, saves the file through `lib/storage.js`, creates the request, and notifies the module's teachers. A teacher, admin or management user opens `GET /api/evidence?status=pending`, and approves with `PATCH /api/evidence/:id`. `relief.service.review` marks the request approved and sets every session of that module on that day to Present in one transaction, then notifies the student. Because the percentage is always calculated from attendance rows, the student's percentage rises immediately with no extra step.

12.5 Worked example: the low-attendance alert

The admin presses the alert button, which calls `POST /api/admin/jobs/low-attendance`. `admin.service.runLowAttendanceJob` loads all enrolments, asks `stats.service` for each student's percentage per module, and for each one below 90 checks whether the same module was already mentioned to that student in the last 7 days. If not, `notifications.service.notify` saves a message and sends an email. The admin sees how many were scanned, alerted and skipped, and the run is added to the audit log.

13. Differences from earlier reports

The code in this report is the current backend. Earlier project reports describe a slightly different version, so the report text and the progress report should be aligned with the table below.

Topic	Earlier progress report	Current code
Code lifetime	5 minutes	Default 10 minutes from `CODE_VALIDITY_MINUTES`. The teacher request can override it (1 to 120 minutes)
Absent students	A timer every minute saves absent rows when the code expires	No timer. Absence is worked out when numbers are read: a held session with no attendance

Topic	Earlier progress report	Current code
		row is absent
Flag reasons	Five (location not shared, browser could not get location, weak GPS, outside area, sent late)	Three: `permission_denied`, `outside_radius`, `low_accuracy`. The check-in does not accept `locationError` or `offline` fields
Backend structure	One large `server.js` with numbered sections	A layered `src/` folder (routes, controllers, services and so on). `server.js` only starts the app
Evidence and module	Module optional, approval covers all of a student's classes that day	Module is required, and approval covers that module's sessions on that day
Percentage wording	"Credit-based" in `API.md`	The calculation is hour-weighted. It is not the removed credit system, and the wording should say so
Notifications	The project notes say the Notification entity was removed from the database design	A `notifications` table and service still exist and are used by many features
Admin	The frontend report says the Admin area was removed from the frontend	The backend still has the full set of admin routes
Other-branch features	The database and contribution report lists departments, rooms, timetable, a self-enrolment switch and a student-removed notification	None of these are in this codebase

These need a team decision on which version is final. Until then this report describes only what the code in this branch does.

14. Known limitations and next steps

14.1 Limitations found in the code

- **Classroom centre comes from the teacher's phone.** If the teacher's GPS is poor when the code is generated, every student is measured from a wrong centre.

- **Indoor GPS accuracy.** The default radius is 10 m and anything with accuracy worse than 50 m is flagged. Both need a test in a real classroom, and a wider radius (25 to 50 m) may be needed.
- **The `Late` status is never produced.** It counts as attended, but only seed data uses it. Nothing in the code decides that a check-in is late.
- **No scheduler** for the low-attendance alert. It runs only when an admin triggers it.
- **Logins are not revoked on password change.** Existing tokens stay valid until they expire (8 hours), unless the account is deactivated.
- **Placeholder secret in development.** If `JWT_SECRET` is missing outside production, an insecure development secret is used, and a warning is printed.
- **`set-test-passwords.js` changes every user.** It should never be run on the shared database.
- **`multer` 1.x has known vulnerabilities.** An upgrade should be planned.
- **Rate limits are kept in server memory.** They reset on restart and are not shared if several servers run.
- **Times are in UTC.** "Today" on the teacher dashboard uses UTC midnight.
- **No automated tests.** `package.json` has no test script.
- **Small inconsistency.** A duplicate enrolment key returns 403 `KEY_TAKEN`, where 409 would be more usual.

14.2 Open question

Flagged check-ins currently do not count towards attendance until a teacher accepts them. The team should confirm that this is the final rule.

14.3 Next steps

1. Agree which branch and which feature set is final, then align the progress report, database report and this report.
2. Write and run a test plan: functional tests for each route, role-based access tests for all four roles, concurrent check-ins, and a real-classroom geofence test.
3. Decide whether to add an automatic schedule for the low-attendance alert and whether a late status is needed.
4. Build the teacher screen for flagged check-ins on the frontend, using the existing accept and reject routes.
5. Connect the management screens to the real management endpoints.
6. Before deployment: set a strong `JWT_SECRET`, change the database password, set `FRONTEND_URL`, configure Supabase Storage, and set `NODE_ENV=production` on Render.
7. Upgrade `multer`, and add an automated test script.